

# **DETECÇÃO DE REVIEWS FALSAS/SINTÉTICAS EM PORTUGUÊS BRASILEIRO USANDO CNN 1D**

**Estudante:** Randerson Sousa de Sá Nunes

**Professor:** Prof. Dr. Rafael Torres Anchieta

**Disciplina:** Aprendizado Profundo

**Data:** Julho de 2026

**Repositório do projeto:**

<<https://github.com/Altaeir-13/Indago-PT>>

# 1 Introdução

As avaliações publicadas em plataformas de comércio eletrônico exercem influência direta sobre a confiança dos consumidores, a reputação de lojas e a decisão de compra. Nesse contexto, a presença de avaliações artificiais, enganosas ou sintéticas pode comprometer a percepção pública sobre produtos e serviços. Este trabalho investiga um recorte específico desse problema: a classificação textual de avaliações individuais em português brasileiro como genuínas ou falsas/sintéticas.

A tarefa de aprendizado abordada é uma classificação binária supervisionada. A entrada do sistema é o texto de uma avaliação e a saída é um rótulo numérico, em que 0 representa uma avaliação falsa/sintética e 1 representa uma avaliação genuína. O modelo principal utilizado é uma rede neural convolucional unidimensional, implementada em PyTorch, comparada com dois modelos de referência baseados em TF-IDF: Regressão Logística e SVM Linear.

A escolha da CNN 1D é motivada pela capacidade desse tipo de arquitetura de capturar padrões locais em sequências textuais, como combinações curtas de palavras e expressões recorrentes. Trata-se de uma arquitetura adequada para demonstrar um pipeline completo de aprendizado profundo, mantendo menor complexidade computacional em comparação com modelos contextuais pré-treinados, como BERTimbau (SOUZA; NOGUEIRA; LOTUFO, 2020).

A escolha pela CNN 1D, em vez de uma arquitetura recorrente, também se justifica pelo tipo de tarefa investigada. Embora RNNs, LSTMs e GRUs sejam historicamente associadas ao processamento sequencial, a classificação de reviews individuais não exige necessariamente a modelagem autorregressiva de dependências longas. Neste trabalho, as avaliações são relativamente curtas e a decisão de classificação pode depender fortemente de padrões locais de palavras, expressões avaliativas e combinações recorrentes no texto. Nesse cenário, filtros convolucionais unidimensionais funcionam como detectores de padrões locais sobre a sequência de tokens, enquanto o Global Max Pooling seleciona os sinais mais relevantes independentemente da posição em que aparecem na avaliação. Essa escolha encontra apoio em Kim (2014), que demonstrou que CNNs simples aplicadas sobre embeddings podem alcançar desempenho competitivo em tarefas de classificação de sentenças, e em Bai, Kolter e Koltun (2018), que argumentaram que a associação automática entre sequências e redes recorrentes deve ser reconsiderada, mostrando que arquiteturas convolucionais temporais podem ser alternativas fortes a LSTMs e GRUs em diferentes tarefas sequenciais. Portanto, a CNN 1D foi adotada neste trabalho como uma alternativa simples, eficiente, paralelizável e adequada ao objetivo de classificação textual supervisionada.

## 2 Apresentação do Dataset

O conjunto de dados utilizado neste trabalho foi o *Fake Reviews PT-BR Dataset*, disponibilizado publicamente por Borges et al. (2025) e por seu repositório de dados (BORGES, 2025). Esse corpus foi construído a partir de avaliações genuínas extraídas da base pública *Brazilian*

*E-Commerce Public Dataset by Olist* (SIONEK, 2018) e de avaliações falsas/sintéticas geradas por GPT-2 em português brasileiro. O dataset utilizado no experimento contém avaliações textuais, categorias de produtos e rótulos binários associados à distinção entre reviews genuínas e falsas/sintéticas.

O repositório público do *Fake Reviews PT-BR Dataset* está disponível em <<https://github.com/cristianomg10/fake-reviews-ptbr-dataset>>. A base original Olist está disponível em <<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>>. No experimento, foi utilizado o corpus textual disponibilizado, contendo os campos lógicos descritos na Tabela 1.

**Tabela 1:** Campos do dataset utilizado

| Campo lógico       | Nome da coluna                | Descrição                              |
|--------------------|-------------------------------|----------------------------------------|
| Texto da avaliação | <i>review_comment_message</i> | Conteúdo textual da review             |
| Categoria          | <i>product_category_name</i>  | Categoria do produto                   |
| Rótulo             | <i>label</i>                  | 0 para falsa/sintética; 1 para genuína |

Após a etapa de limpeza mínima, o conjunto de dados considerado no experimento totalizou 29.988 amostras. A classe 0 corresponde a avaliações falsas/sintéticas e a classe 1 corresponde a avaliações genuínas. Os dados são textos em português brasileiro, organizados em formato tabular, acompanhados por categoria de produto e rótulo supervisionado.

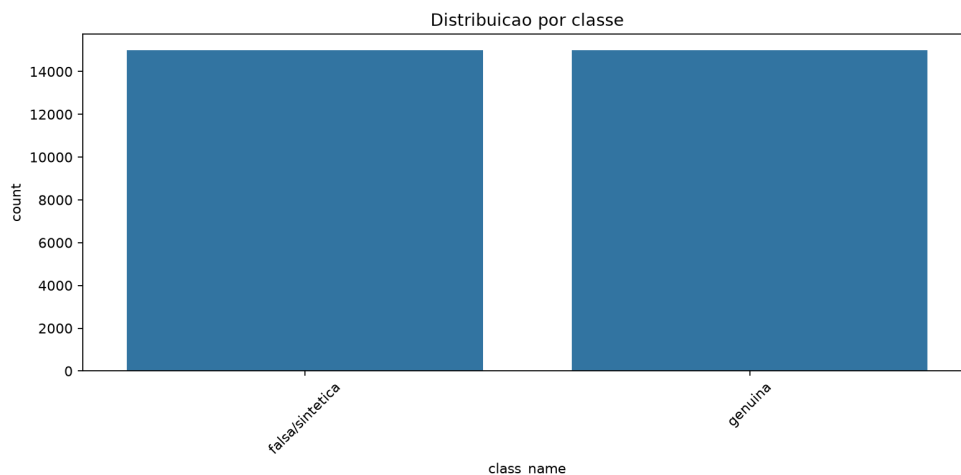
A entrada textual pode ser representada formalmente como:

$$X = (x_1, x_2, \dots, x_T),$$

em que  $T$  corresponde ao tamanho da sequência textual e  $x_t$  representa o token observado na posição  $t$ . Antes de ser processado pela rede neural, o texto é tokenizado, truncado ou preenchido por padding e convertido em índices de vocabulário.

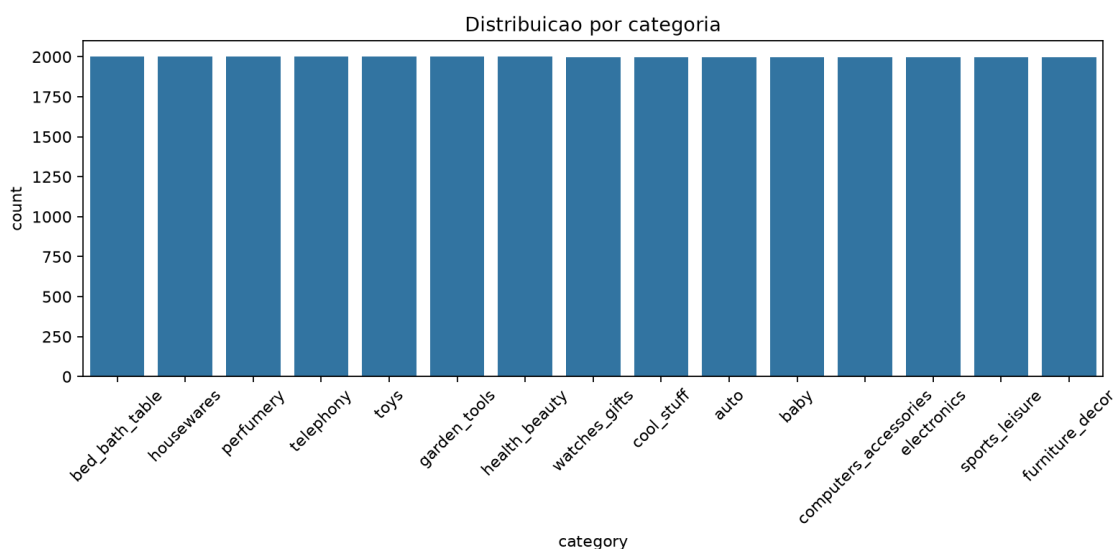
**Tabela 2:** Exemplos curtos de avaliações do dataset

| Classe              | Categoria       | Exemplo curto                                    |
|---------------------|-----------------|--------------------------------------------------|
| 0 - falsa/sintética | furniture_decor | “tudo perfeito. Ainda não recebi meu produto...” |
| 1 - genuína         | housewares      | “produto entregue antes do prazo”                |



**Figura 1:** Distribuição das classes no dataset após a limpeza

Fonte: elaborado pelo autor com base nos resultados do experimento.



**Figura 2:** Distribuição das categorias de produto no dataset

Fonte: elaborado pelo autor com base nos resultados do experimento.

### 3 Limitações e Tratamento do Dataset

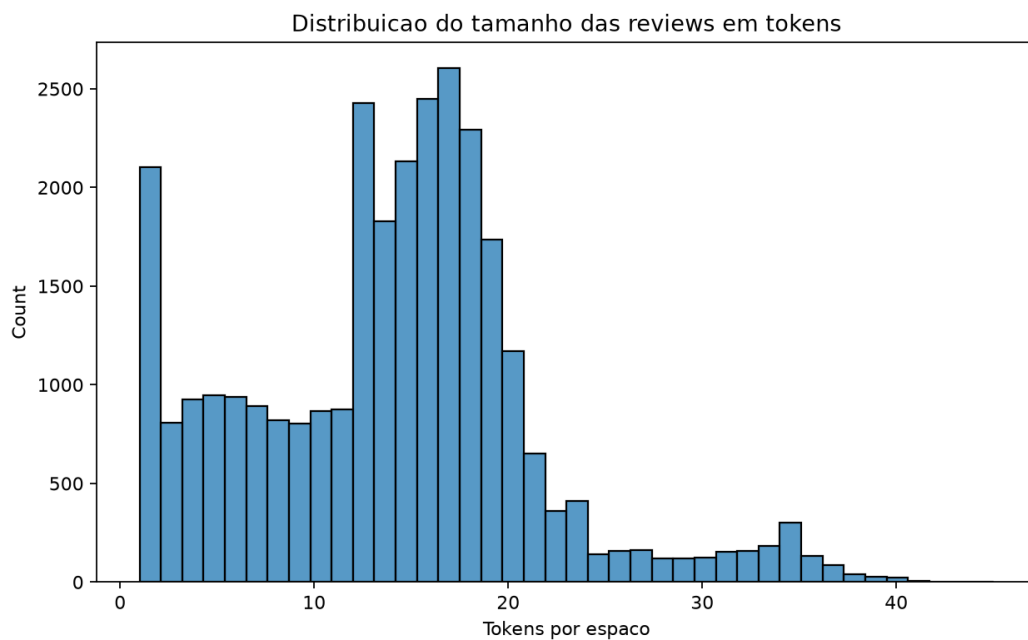
A limpeza aplicada aos textos foi propositalmente mínima, com o objetivo de preservar sinais linguísticos potencialmente úteis para a classificação. Foram removidas avaliações vazias, e foram normalizados espaços consecutivos e quebras de linha. Não foi realizada remoção agressiva de pontuação, acentos ou marcas lexicais, pois tais elementos podem contribuir para a distinção entre os padrões textuais das classes.

A análise exploratória indicou ausência de valores nulos nas colunas canônicas após o carregamento, mas identificou 683 linhas duplicadas e 1.537 textos duplicados. Também se

observou que as avaliações são relativamente curtas, com média de 82,95 caracteres e 13,98 tokens por review, além de mediana de 15 tokens.

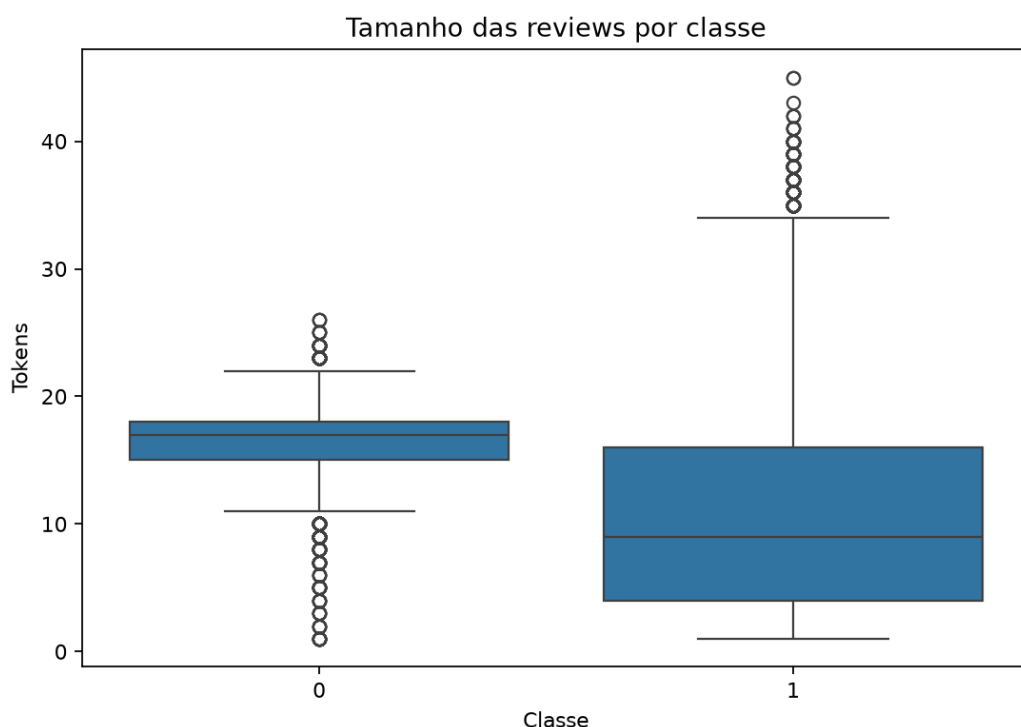
A análise exploratória identificou duplicatas no conjunto de dados. Como o protocolo principal manteve o dataset após a limpeza mínima de textos vazios e normalização de espaços, a presença de textos duplicados pode representar uma ameaça à validade experimental, pois exemplos idênticos ou muito semelhantes podem aparecer em subconjuntos distintos de treino, validação e teste. Assim, os resultados devem ser interpretados com cautela. Como trabalho futuro, recomenda-se executar uma avaliação adicional com deduplicação antes do split, a fim de medir o impacto dessa decisão sobre as métricas.

A principal limitação conceitual do dataset está relacionada à natureza da classe falsa. Essa classe é sintética e operacional, pois foi gerada por GPT-2, e não representa necessariamente fraude humana comprovada. Além disso, o conjunto de dados não contém usuário, data, rating, produto específico ou relações entre contas. Portanto, este trabalho não realiza detecção completa de astroturfing, limitando-se à classificação de avaliações individuais.



**Figura 3:** Distribuição do tamanho das avaliações em tokens

Fonte: elaborado pelo autor com base nos resultados do experimento.



**Figura 4:** Distribuição do tamanho das avaliações por classe

Fonte: elaborado pelo autor com base nos resultados do experimento.

## 4 Trabalhos Relacionados

Borges et al. (2025) apresentam um benchmark de algoritmos de aprendizado de máquina para detecção de fake reviews em português brasileiro. Esse trabalho é diretamente relacionado ao presente estudo por tratar do mesmo domínio e por disponibilizar uma base pública adequada à investigação de avaliações falsas/sintéticas.

Kim (2014) propôs o uso de redes neurais convolucionais para classificação de sentenças, demonstrando que filtros convolucionais aplicados sobre embeddings podem capturar padrões discriminativos em texto. Essa referência fundamenta a escolha da CNN 1D como arquitetura principal deste trabalho.

Souza, Nogueira e Lotufo (2020) introduziram o BERTimbau, um modelo contextual pré-treinado para português brasileiro. Embora modelos baseados em BERT sejam relevantes para trabalhos futuros, eles não foram implementados neste experimento, pois o objetivo da avaliação foi treinar e analisar diretamente uma CNN 1D. Devlin et al. (2019) e Goodfellow, Bengio e Courville (2016) também são referências importantes para contextualizar modelos Transformer e fundamentos de aprendizado profundo. As bibliotecas utilizadas no experimento dialogam com ferramentas consolidadas de aprendizado de máquina e deep learning, como scikit-learn (PEDREGOSA et al., 2011) e PyTorch (PASZKE et al., 2019).

## 5 Método Implementado

O método implementado segue um pipeline textual composto pelas seguintes etapas: texto da avaliação, tokenização, padding ou truncamento, embedding treinável, convolução unidimensional, função de ativação ReLU, Global Max Pooling, dropout, camada linear e saída binária. A camada de embedding converte tokens discretos em vetores densos. A convolução unidimensional aplica filtros locais sobre a sequência, buscando padrões curtos de palavras. A função ReLU introduz não linearidade ao modelo, enquanto o Global Max Pooling seleciona os sinais mais intensos detectados pelos filtros.

O dropout foi utilizado como mecanismo de regularização para reduzir overfitting. A camada linear final gera um logit, que é usado diretamente pela função BCEWithLogitsLoss durante o treinamento. Essa função combina a sigmoid e a entropia cruzada binária de forma numericamente estável. Na avaliação, a sigmoid é aplicada ao logit para obter a probabilidade associada à classe genuína, e a probabilidade da classe falsa/sintética é calculada de forma complementar. A otimização foi feita com Adam, e o treinamento utilizou early stopping para interromper a execução quando a validação deixava de melhorar.

**Tabela 3:** Arquitetura da CNN 1D implementada

| Camada               | Configuração                                             | Função                                        |
|----------------------|----------------------------------------------------------|-----------------------------------------------|
| Embedding            | vocab_size=30000,<br>embedding_dim=128,<br>padding_idx=0 | Representar tokens como vetores densos.       |
| Conv1D               | 256 filtros, kernel_size=7                               | Capturar padrões locais da sequência textual. |
| ReLU                 | Sem parâmetros treináveis                                | Introduzir não linearidade.                   |
| Global Max Pooling   | Agregação máxima global na sequência                     | Selecionar ativações mais relevantes.         |
| Dropout              | p=0.5                                                    | Reduzir overfitting.                          |
| Linear               | 256 $\rightarrow$ 1                                      | Gerar o logit da classificação binária.       |
| Sigmoid na avaliação | Aplicada após a saída linear                             | Converter o logit em probabilidade.           |

Cabe destacar que nem todas as operações da arquitetura possuem parâmetros treináveis. Camadas como ReLU e Global Max Pooling consistem em operações fixas, enquanto a Sigmoid atua apenas como uma transformação matemática na saída durante a fase de inferência.

No método forward, a sequência de índices passa inicialmente pela camada de embedding,

produzindo uma matriz com um vetor denso por token. Em seguida, essa matriz é transposta para o formato esperado pela convolução unidimensional, com os canais correspondendo às dimensões do embedding. A convolução e a ReLU produzem mapas de ativação ao longo da sequência; o Global Max Pooling seleciona, para cada filtro, a ativação máxima observada. O vetor resultante passa pelo dropout e pela camada linear, que retorna um único logit por avaliação. As principais camadas da arquitetura foram instanciadas utilizando os módulos nativos padrão do ambiente PyTorch.

## **6 Configuração Experimental**

O experimento foi desenvolvido em Python. A implementação da CNN 1D foi realizada com PyTorch, enquanto os modelos de referência foram treinados com scikit-learn. As bibliotecas pandas e numpy foram utilizadas para manipulação dos dados, e matplotlib e seaborn foram empregadas na geração dos gráficos.

O código-fonte, as instruções de reprodução e a descrição das dependências estão disponíveis no repositório do projeto na documentação principal. A entrega inclui o relatório final, o código-fonte no GitHub, os slides da apresentação e a referência ao dataset utilizado. O repositório do projeto está disponível em <<https://github.com/Altaeir-13/Indago-PT-Detecting-Synthetic-Reviews-in-Brazilian-Portuguese>>. A reprodução completa está documentada no repositório, incluindo instalação das dependências, posicionamento do dataset, execução do procedimento experimental e descrição dos artefatos gerados.

A divisão dos dados foi estratificada em treino, validação e teste, nas proporções de 70%, 15% e 15%, respectivamente, com random\_state igual a 13. Os modelos avaliados foram TF-IDF com Regressão Logística, TF-IDF com SVM Linear e CNN 1D em PyTorch. As métricas foram calculadas no conjunto de teste.

### **6.1 Ambiente de execução**

O experimento foi executado localmente com aceleração por GPU, utilizando uma NVIDIA GeForce RTX 3060 Ti. No ambiente Python utilizado, o PyTorch 2.11.0+cu128 reconheceu CUDA 12.8 e selecionou o dispositivo cuda para o treinamento da CNN 1D.



**Tabela 4:** Ambiente de execução registrado

| Item                                   | Valor                      |
|----------------------------------------|----------------------------|
| Sistema operacional                    | Windows 11 10.0.26200-SP0  |
| Python                                 | 3.14.5                     |
| PyTorch                                | 2.11.0+cu128               |
| scikit-learn                           | 1.9.0                      |
| pandas                                 | 3.0.3                      |
| numpy                                  | 2.5.0                      |
| matplotlib                             | 3.11.0                     |
| seaborn                                | 0.13.2                     |
| GPU detectada via CUDA                 | Sim                        |
| Modelo da GPU                          | NVIDIA GeForce RTX 3060 Ti |
| <code>torch.cuda.is_available()</code> | True                       |
| Dispositivo utilizado no treinamento   | cuda                       |

**Tabela 5:** Hiperparâmetros principais da CNN 1D

| Parâmetro                  | Valor             |
|----------------------------|-------------------|
| <code>vocab_size</code>    | 30000             |
| <code>max_len</code>       | 256               |
| <code>embedding_dim</code> | 128               |
| <code>filters</code>       | 256               |
| <code>kernel_size</code>   | 7                 |
| <code>dropout</code>       | 0.5               |
| <code>lr</code>            | 0.001             |
| <code>batch_size</code>    | 128               |
| <code>epochs</code>        | 50                |
| <code>patience</code>      | 3                 |
| <code>loss</code>          | BCEWithLogitsLoss |
| <code>optimizer</code>     | Adam              |
| <code>random_state</code>  | 13                |

Neste experimento, foi adotada uma configuração única de hiperparâmetros, definida com base em valores usuais para modelos CNN 1D em classificação textual e na viabilidade computacional da avaliação. O melhor checkpoint da CNN foi selecionado com base na menor perda de validação, por meio de early stopping.

**Tabela 6:** Modelos avaliados no experimento

| Modelo                       | Representação                | Finalidade                        |
|------------------------------|------------------------------|-----------------------------------|
| TF-IDF + Regressão Logística | TF-IDF                       | Baseline linear probabilístico    |
| TF-IDF + SVM Linear          | TF-IDF                       | Baseline linear de margem         |
| CNN 1D PyTorch               | Tokens e embedding treinável | Modelo principal de deep learning |

Foram utilizadas as seguintes métricas: acurácia, precisão para a classe falsa/sintética, recall para a classe falsa/sintética, F1-score para a classe falsa/sintética, F1 macro, AUC-ROC orientada para a classe 0 e matriz de confusão. A classe de interesse foi definida como label 0, pois representa a avaliação falsa/sintética.

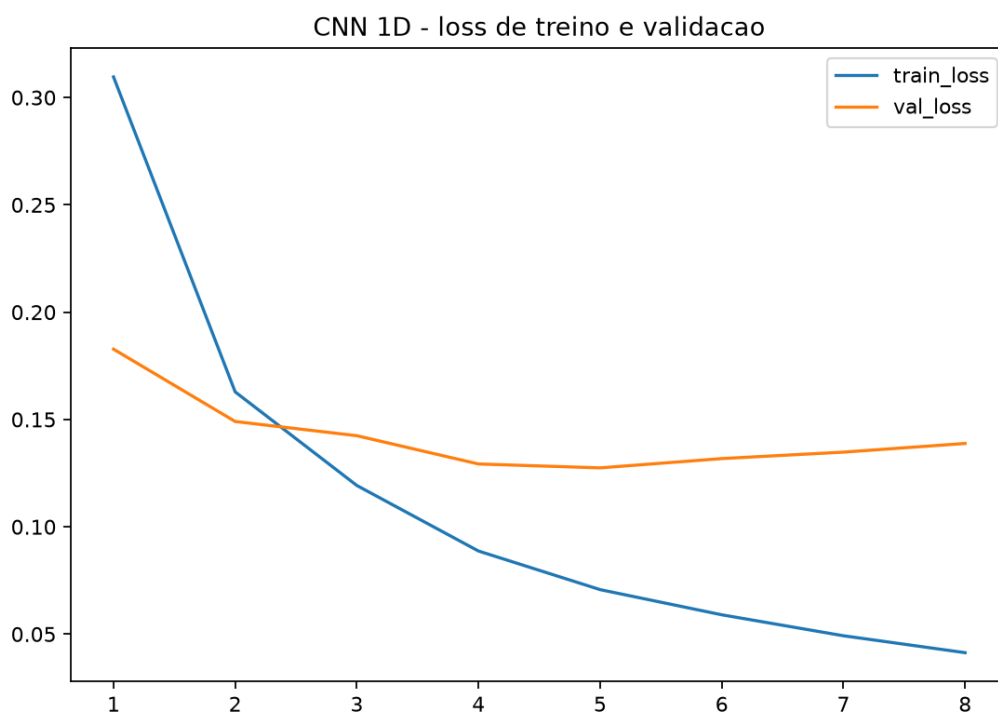
## 7 Resultados

A Tabela 7 apresenta os resultados reais do experimento no conjunto de teste. A CNN 1D em PyTorch obteve o melhor desempenho geral, superando os dois modelos de referência baseados em TF-IDF.

**Tabela 7:** Resultados quantitativos no conjunto de teste

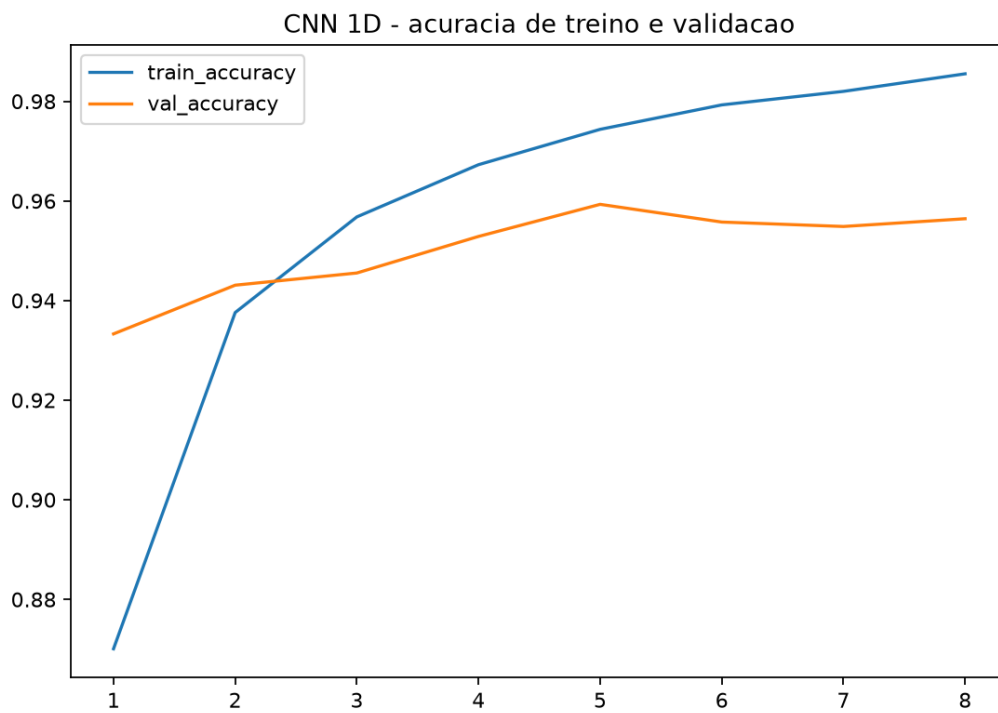
| Modelo                       | Accuracy | Precision 0 | Recall 0 | F1 0     | F1 macro | AUC-ROC 0 |
|------------------------------|----------|-------------|----------|----------|----------|-----------|
| TF-IDF + Regressão Logística | 0.877084 | 0.855765    | 0.907111 | 0.880690 | 0.876971 | 0.947198  |
| TF-IDF + SVM Linear          | 0.897311 | 0.878814    | 0.921778 | 0.899783 | 0.897248 | 0.960960  |
| CNN 1D PyTorch               | 0.961325 | 0.973540    | 0.948444 | 0.960828 | 0.961319 | 0.991352  |

A CNN 1D atingiu 0,961325 de acurácia, 0,960828 de F1-score para a classe falsa/sintética, 0,961319 de F1 macro e 0,991352 de AUC-ROC orientada para a classe 0. Entre os baselines, o SVM Linear apresentou desempenho superior ao da Regressão Logística.



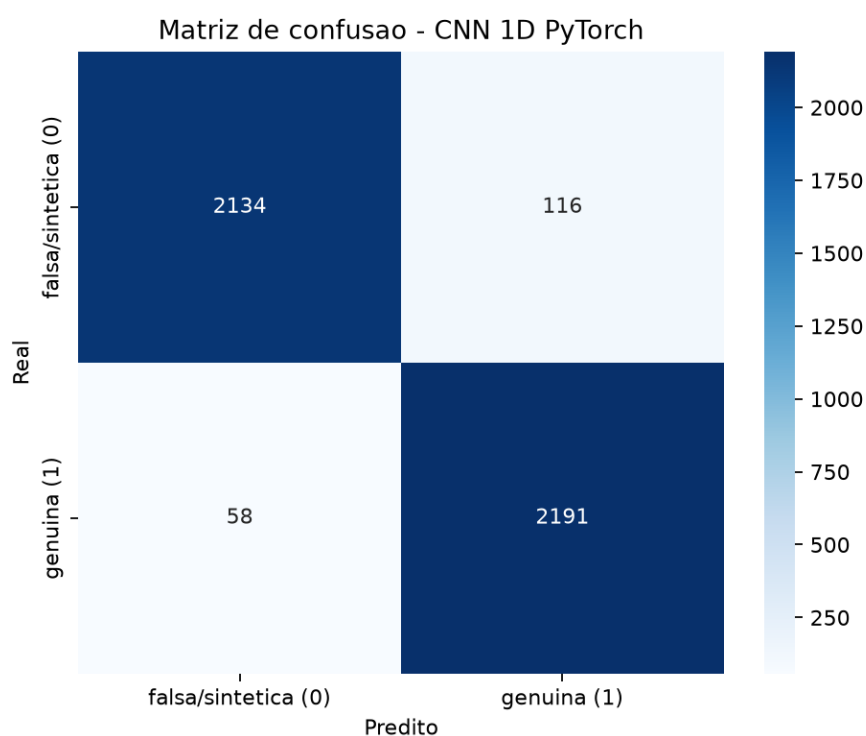
**Figura 5:** Curva de perda da CNN 1D durante o treinamento

Fonte: elaborado pelo autor com base nos resultados do experimento.



**Figura 6:** Curva de acurácia da CNN 1D durante o treinamento

Fonte: elaborado pelo autor com base nos resultados do experimento.



**Figura 7:** Matriz de confusão da CNN 1D no conjunto de teste

Fonte: elaborado pelo autor com base nos resultados do experimento.

## 8 Análise e Discussão

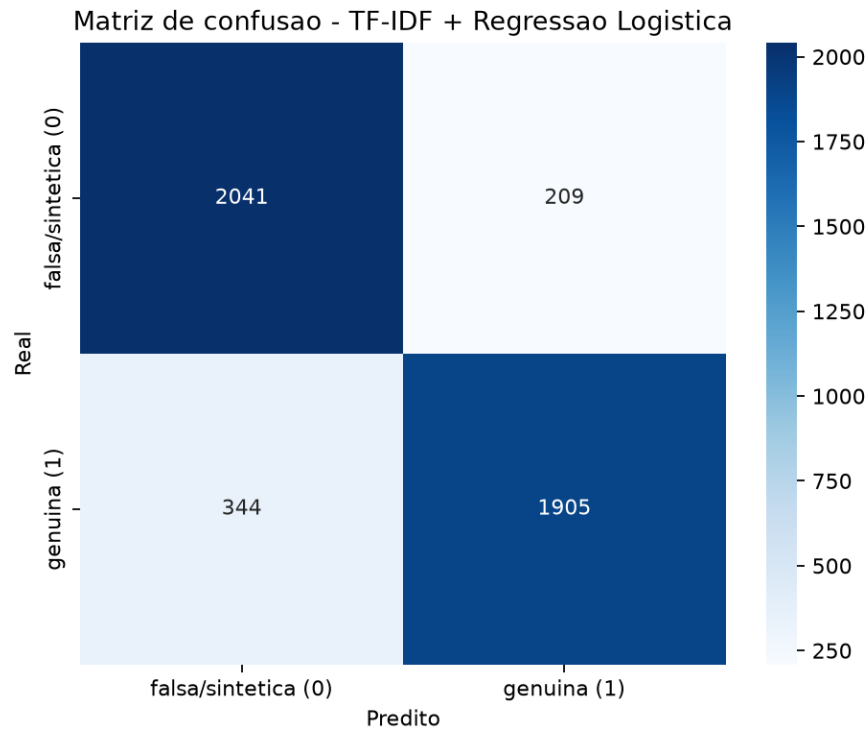
A CNN 1D apresentou o melhor desempenho geral do experimento. Em comparação com o SVM Linear, a CNN obteve ganho aproximado de 6,4 pontos percentuais de acurácia. Esse resultado sugere que o uso de embeddings treináveis e filtros convolucionais foi eficaz para capturar padrões locais relevantes nas avaliações textuais.

O SVM Linear foi o baseline clássico mais forte, indicando que representações TF-IDF continuam competitivas em tarefas de classificação textual. A Regressão Logística também apresentou desempenho consistente, mas inferior ao SVM Linear e à CNN 1D.

A AUC-ROC elevada da CNN, aproximadamente 99,14%, indica boa separação entre reviews sintéticas e genuínas no conjunto de teste. O F1 macro da CNN foi aproximadamente 96,13%, sugerindo desempenho equilibrado entre as classes. As curvas de treinamento e validação indicam que o modelo aprendeu padrões relevantes do conjunto de treino sem apresentar sinais fortes de underfitting. A menor perda de validação ocorreu na época 5; depois disso, a perda de treino continuou diminuindo, enquanto a perda de validação oscilou e aumentou levemente. Essa diferença sugere possibilidade de overfitting leve, controlado parcialmente pelo uso de dropout e early stopping. O treinamento foi encerrado na época 8, após três épocas sem melhora suficiente na perda de validação, e o melhor checkpoint foi escolhido com base na perda de validação. A matriz de confusão da CNN reforça a interpretação de que o modelo apresentou

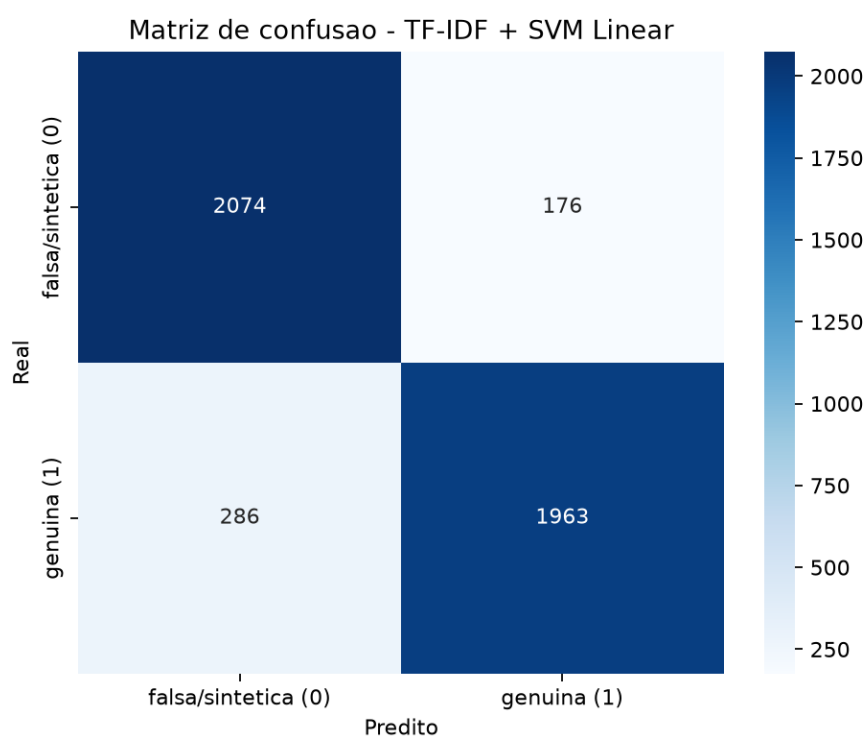
desempenho equilibrado entre as classes.

Apesar dos resultados positivos, a interpretação deve ser cautelosa. A classe falsa foi gerada por GPT-2, o que significa que o modelo pode ter aprendido padrões específicos do gerador ou do processo de construção do dataset. Assim, os resultados não devem ser interpretados como detecção de fraude humana real ou de astroturfing completo.



**Figura 8:** Matriz de confusão do baseline TF-IDF + Regressão Logística

Fonte: elaborado pelo autor com base nos resultados do experimento.



**Figura 9:** Matriz de confusão do baseline TF-IDF + SVM Linear

Fonte: elaborado pelo autor com base nos resultados do experimento.

## 9 Análise Qualitativa de Erros

A análise qualitativa de erros foi realizada a partir das predições da CNN 1D no conjunto de teste. Falsos positivos correspondem a avaliações genuínas classificadas como falsas/sintéticas. Falsos negativos correspondem a avaliações falsas/sintéticas classificadas como genuínas.

**Tabela 8:** Exemplos curtos de erros da CNN 1D

| Tipo de erro   | Interpretação                        | Exemplo curto                                                              |
|----------------|--------------------------------------|----------------------------------------------------------------------------|
| Falso positivo | Genuína predita como falsa/sintética | “Muito ruim ... Comprei 02 Kits e foi entregue somente 1 Kit de cabide...” |
| Falso negativo | Falsa/sintética predita como genuína | “Tudo como anunciado... Ótimo produto...”                                  |

Esses casos sugerem que avaliações muito curtas, genéricas, padronizadas ou com poucos detalhes concretos podem ficar próximas da fronteira de decisão do modelo. Também é possível que alguns textos sintéticos imitem padrões reais das avaliações da Olist, dificultando a separação exclusivamente com base em características textuais.

A análise registra os exemplos incorretamente classificados para revisão posterior. A inclusão sistemática de exemplos corretamente classificados é uma melhoria futura de documentação.

## 10 Comparação com Trabalhos Relacionados

A comparação com a literatura deve considerar que os protocolos, conjuntos de dados, métricas e objetivos não são idênticos. Ainda assim, a Tabela 9 posiciona o presente trabalho em relação a referências relevantes da área, incluindo estudos internacionais que também utilizam CNN para detecção de fake reviews, o que fortalece a justificativa da escolha arquitetural deste trabalho.

Quando os protocolos não são diretamente comparáveis, a tabela apresenta o resultado reportado no trabalho original e explicita a limitação da comparação.

Borges et al. (2025) avaliaram combinações de classificadores tradicionais com representações BoW, TF-IDF e BERTimbau para fake reviews em português brasileiro. No artigo, o melhor resultado foi obtido com Regressão Logística usando BERTimbau: 96,55% de acurácia e 96,54% de F1-score em validação cruzada estratificada com 10 folds; no holdout com 30% para teste, a mesma combinação atingiu 96,61% de acurácia, 96,62% de F1-score e 141 falsos positivos. Esses valores são reportados como referência direta de domínio e dataset, mas não constituem uma comparação controlada com este trabalho, pois o presente experimento usa outro protocolo, outro split e outra arquitetura.

Para contextualizar a CNN 1D como escolha arquitetural, Deshai e Rao (2023) propuseram um método que combina CNN com *Adaptive Particle Swarm Optimization* (APSO) para detecção de fake reviews em cinco datasets internacionais (Ott, Amazon, Yelp, TripAdvisor e IMDb), atingindo 99,4% de acurácia e superando classificadores clássicos de aprendizado de máquina. É relevante notar que os autores relatam hiperparâmetros muito próximos aos utilizados neste trabalho, como *kernel\_size* igual a 7, *batch\_size* igual a 128 e otimizador Adam com taxa de aprendizado 0,001, o que reforça que as escolhas de configuração adotadas seguem práticas já validadas na literatura internacional para CNNs aplicadas a texto. No mesmo artigo, os autores citam o estudo de Vidanagama, Silva e Karunananda (2020), no qual uma CNN sem otimização adicional atingiu 92,7% de acurácia no dataset Yelp e 97,3% no dataset Amazon, evidenciando que arquiteturas convolucionais relativamente simples já obtêm desempenho competitivo nessa tarefa antes mesmo de otimizações adicionais.

Esses dois trabalhos internacionais mostram que a CNN é uma arquitetura estabelecida e frequentemente utilizada para classificação de fake reviews, o que sustenta a escolha metodológica deste trabalho. Contudo, cabe destacar que os datasets, idiomas e protocolos de avaliação usados por Deshai e Rao (2023) e por Vidanagama, Silva e Karunananda (2020) diferem do dataset Fake Reviews PT-BR utilizado neste experimento, de modo que os valores numéricos não podem ser comparados diretamente com os resultados obtidos aqui; eles servem como evidência

de que a arquitetura escolhida é consistente com a literatura, e não como benchmark direto de desempenho.

**Tabela 9:** Comparação objetiva com trabalhos relacionados

| Trabalho                               | Dataset                               | Modelo                          | Resultado principal                                                                                                            |
|----------------------------------------|---------------------------------------|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Borges et al. (2025)                   | Fake Reviews PT-BR                    | Regressão Logística + BERTimbau | 96,55% de acurácia e 96,54% de F1-score em 10-fold; no holdout, 96,61% de acurácia, 96,62% de F1-score e 141 falsos positivos. |
| Vidanagama, Silva e Karunananda (2020) | Yelp e Amazon                         | CNN para texto                  | 92,7% de acurácia no Yelp e 97,3% no Amazon, conforme reportado por Deshai e Rao (2023).                                       |
| Deshai e Rao (2023)                    | Ott, Amazon, Yelp, TripAdvisor e IMDb | CNN + Adaptive PSO              | 99,4% de acurácia, superando classificadores clássicos de aprendizado de máquina.                                              |
| Kim (2014)                             | Classificação de sentenças em inglês  | CNN para texto                  | Referência arquitetural para CNNs em texto; sem comparação numérica direta com o dataset deste trabalho.                       |
| Este trabalho                          | Fake Reviews PT-BR                    | CNN 1D PyTorch                  | 0,961319 de F1 macro e 0,991352 de AUC-ROC para a classe falsa/sintética.                                                      |

*Observação:* os resultados da literatura não são diretamente comparáveis aos deste trabalho, pois diferem em idioma, dataset, protocolo experimental, representação textual e métricas de avaliação.

Em síntese, a CNN 1D deste trabalho obteve resultado próximo ao estado da arte reportado por Borges et al. (2025) para o dataset em português brasileiro, mesmo utilizando uma arquitetura mais simples e computacionalmente mais barata do que um modelo baseado em Transformer. Ao mesmo tempo, os resultados internacionais de Deshai e Rao (2023) e Vidanagama, Silva e Karunananda (2020) indicam que arquiteturas convolucionais têm se mostrado consistentemente eficazes para esse tipo de tarefa em diferentes idiomas e domínios, o que reforça a adequação metodológica da escolha realizada neste experimento.

## 11 Conclusão

Este trabalho apresentou um experimento completo para classificação de reviews falsas/sintéticas em português brasileiro usando uma CNN 1D em PyTorch. O pipeline incluiu carregamento do dataset, limpeza mínima, análise exploratória, divisão estratificada, treinamento de baselines, treinamento do modelo neural, avaliação quantitativa e análise qualitativa de erros.



A CNN 1D superou os baselines avaliados, atingindo 0,961325 de acurácia e 0,961319 de F1 macro. Esses resultados indicam que uma arquitetura convolucional simples pode ser competitiva para a classificação textual neste dataset. Contudo, as conclusões devem permanecer restritas ao escopo do experimento: o modelo distingue avaliações genuínas de avaliações sintéticas geradas no dataset, mas não comprova detecção de fraude humana nem de astroturfing completo.

Como continuidade, o projeto pode ser reaproveitado em um Trabalho de Conclusão de Curso como pipeline experimental reutilizável, como etapa intermediária entre modelos TF-IDF e BERTimbau, e como base para discussão de ameaças à validade. Trabalhos futuros podem incluir comparação com BERTimbau, CNN multi-kernel, validação cruzada, deduplicação antes da divisão dos dados e uso de metadados comportamentais ou grafos quando houver dados apropriados.

## 12 Declaração de Uso de IA Generativa

Foi utilizada ferramenta de IA generativa, como ChatGPT/Codex, como apoio na organização textual, revisão de clareza e estruturação do relatório. Os experimentos, métricas, tabelas e gráficos apresentados foram obtidos por execução local do código-fonte do projeto. A correção das informações foi verificada por inspeção do código, conferência dos resultados gerados, comparação entre tabelas de métricas e saídas dos experimentos, além de revisão manual do texto final. O autor assume responsabilidade pela implementação, análise dos resultados e conteúdo final apresentado.

## Referências

BAI, S.; KOLTER, J. Z.; KOLTUN, V. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. In: PMLR. *Proceedings of the 35th International Conference on Machine Learning*. [S.l.], 2018. p. 686–695.

BORGES, E. C. R. *Fake Reviews PT-BR Dataset*. GitHub, 2025. Disponível em: <<https://github.com/cristianomg10/fake-reviews-ptbr-dataset>>. Acesso em: 2 jul. 2026.

BORGES, E. C. R. et al. Benchmarking machine learning algorithms in fake reviews detection in brazilian portuguese. *Revista Brasileira de Computação Aplicada*, 2025.

DESHAI, N.; RAO, B. B. Unmasking deception: a cnn and adaptive pso approach to detecting fake online reviews. *Soft Computing*, p. 1–22, 2023.

DEVLIN, J. et al. Bert: Pre-training of deep bidirectional transformers for language understanding. In: *Proceedings of NAACL-HLT*. [S.l.: s.n.], 2019.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. [S.l.]: MIT Press, 2016.

KIM, Y. Convolutional neural networks for sentence classification. In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*. [S.l.: s.n.], 2014. p. 1746–1751.

PASZKE, A. et al. Pytorch: An imperative style, high-performance deep learning library. In: *Advances in Neural Information Processing Systems*. [S.l.: s.n.], 2019.

PEDREGOSA, F. et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, v. 12, p. 2825–2830, 2011.

SIONEK, A. *Brazilian E-Commerce Public Dataset by Olist*. Kaggle, 2018. Disponível em: <<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>>. Acesso em: 2 jul. 2026.

SOUZA, F.; NOGUEIRA, R.; LOTUFO, R. Bertimbau: Pretrained bert models for brazilian portuguese. In: *Brazilian Conference on Intelligent Systems*. [S.l.: s.n.], 2020.

VIDANAGAMA, D. U.; SILVA, T. P.; KARUNANANDA, A. S. Deceptive consumer review detection: A survey. *Artificial Intelligence Review*, v. 53, n. 2, p. 1323–1352, 2020.